

Pruebas de Base de Datos

LibretaDigital

¿Cómo se probó la base de datos antes de lanzar la app?

Proyecto	LibretaDigital — Cafetería Escolar
Motor de BD	SQLite 3 (viene integrado en Android)
Herramienta	DB Browser for SQLite + IntelliJ IDEA
Equipo	David Bohórquez (Dev) · Arley Martínez (Analista)

1. ¿Por qué se hicieron estas pruebas?

Antes de escribir una sola línea de código Java en la app, el equipo probó la base de datos por separado. La razón es sencilla: si la base de datos falla (guarda datos incorrectos, permite duplicados, borra cosas que no debe), toda la app falla con ella.

La herramienta usada fue DB Browser for SQLite, que permite escribir y ejecutar comandos SQL directamente, como si fuera una calculadora para bases de datos, sin necesitar el celular ni Android.

Se probaron cinco cosas concretas:

- Que las tablas se crearan bien con las columnas correctas.
- Que los datos se pudieran guardar, modificar y eliminar sin errores.
- Que las consultas devolvieran los datos correctos (los mismos que muestra la app).
- Que el sistema detectara errores como duplicados o deudas negativas.
- Que la base de datos no ocupara demasiado espacio en el celular.

2. Cómo se crearon las tablas

La base de datos de LibretaDigital tiene tres tablas. No se crearon así desde el principio — evolucionaron en varias versiones a medida que se descubrían nuevas necesidades durante las pruebas.

Versión 1 — El punto de partida

La primera versión tenía solo dos tablas básicas: una para guardar el PIN de acceso y otra para los deudores. Sin fechas, sin historial, sin alarmas.

```
-- Tabla para el PIN de acceso
CREATE TABLE App_Auth (
    pin TEXT NOT NULL
);

-- Tabla de deudores (versión inicial)
CREATE TABLE Deudores (
    id_deudor    INTEGER PRIMARY KEY AUTOINCREMENT,
    nombres      TEXT NOT NULL,
    apellidos    TEXT NOT NULL,
    grado        INTEGER NOT NULL,
    grupo        TEXT NOT NULL,
    deuda_actual REAL DEFAULT 0
);
```

💡 En esta versión no había manera de saber cuándo se creó un registro ni qué movimientos había tenido. Eso se detectó como problema en la Sprint Review y se agregó en la siguiente versión.

Versión final (v5) — La que quedó en la app

Tras varias iteraciones, el esquema final agregó el historial de movimientos, las fechas de registro, las alarmas y el código de recuperación de PIN.

```
-- PIN de acceso + código de recuperación
CREATE TABLE App_Auth (
    pin                TEXT NOT NULL,
    pin_recuperacion   TEXT DEFAULT ''
);

-- Deudores con todos los campos necesarios
CREATE TABLE Deudores (
    id_deudor    INTEGER PRIMARY KEY AUTOINCREMENT,
    nombres      TEXT NOT NULL,
    apellidos    TEXT NOT NULL,
    grado        INTEGER NOT NULL,      -- entre 6 y 11
    grupo        TEXT NOT NULL,         -- '01', '02' o '03'
    deuda_actual REAL DEFAULT 0,         -- nunca baja de cero
    fecha_creacion TEXT DEFAULT '',     -- formato: dd/MM/yyyy HH:mm
    alarma_ms    INTEGER DEFAULT -1    -- -1 significa sin alarma
);

-- Historial de cada transacción por deudor
```

```
CREATE TABLE Movimientos (
  id_mov      INTEGER PRIMARY KEY AUTOINCREMENT,
  id_deudor   INTEGER NOT NULL,
  tipo        TEXT NOT NULL,      -- CREACION, CARGO, ABONO o LIQUIDACION
  monto       REAL NOT NULL,
  fecha       TEXT NOT NULL
);
```

3. Datos de prueba insertados

Para probar que todo funcionaba, se insertaron 12 estudiantes ficticios cubriendo todos los grados (6 al 11) y los tres grupos. Se usaron deudas variadas para poder probar los filtros y el ordenamiento.

```
INSERT INTO Deudores (nombres, apellidos, grado, grupo, deuda_actual,
fecha_creacion)
VALUES
('Carlos',      'García Ruiz',      6, '01', 5500, '10/03/2026 09:00'),
('Valeria',     'Mendoza Torres', 6, '02', 12000, '10/03/2026 09:05'),
('Andrés',      'Pérez López',    7, '01', 0, '11/03/2026 10:00'),
('Sofía',       'Ramírez Castro', 7, '03', 32500, '11/03/2026 10:15'),
('Miguel',      'Torres Vega',    8, '01', 8750, '12/03/2026 08:30'),
('Isabela',     'Flores Mora',    8, '02', 15200, '12/03/2026 08:45'),
('Sebastián',   'Vargas Díaz',    9, '01', 47800, '13/03/2026 11:00'),
('Camila',      'Ortiz Herrera',  9, '02', 6100, '13/03/2026 11:20'),
('Julián',      'Castro Moreno', 10, '03', 22300, '14/03/2026 07:45'),
('Mariana',     'Silva Quintero', 10, '01', 3900, '14/03/2026 07:50'),
('Tomás',       'Reyes Gutiérrez', 11, '02', 55000, '15/03/2026 09:10'),
('Valentina',   'Rojas Suárez',  11, '03', 0, '15/03/2026 09:30');
```

4. Consultas probadas — Los mismos reportes de la app

Cada consulta que se ve en la app fue primero probada manualmente en DB Browser. Acá están las más importantes con sus resultados.

Lista principal ordenada A-Z (pantalla Home)

```
SELECT nombres || ' ' || apellidos AS nombre,
       grado || '-' || grupo      AS curso,
       deuda_actual
FROM Deudores
ORDER BY apellidos ASC;
```

Resultado — Lista de deudores orden A-Z

Nombre	Curso	Deuda
Julián Castro Moreno	10-03	\$22.300
Isabela Flores Mora	8-02	\$15.200
Carlos García Ruiz	6-01	\$8.500
Valeria Mendoza Torres	6-02	\$7.000
Camila Ortiz Herrera	9-02	\$6.100
Andrés Pérez López	7-01	\$0
Sofía Ramírez Castro	7-03	\$40.000
Tomás Reyes Gutiérrez	11-02	\$55.000
Valentina Rojas Suárez	11-03	\$0
Mariana Silva Quintero	10-01	\$3.900
Miguel Torres Vega	8-01	\$8.750
Sebastián Vargas Díaz	9-01	\$50.000

Top 5 mayores deudas (filtro 'Mayor deuda')

```
SELECT nombres || ' ' || apellidos AS nombre,
       deuda_actual
FROM Deudores
WHERE deuda_actual > 0
ORDER BY deuda_actual DESC
LIMIT 5;
```

Resultado — Top 5 mayores deudas		
Posición	Nombre	Deuda
1°	Tomás Reyes Gutiérrez	\$55.000
2°	Sebastián Vargas Díaz	\$50.000
3°	Sofía Ramírez Castro	\$40.000
4°	Julián Castro Moreno	\$22.300
5°	Isabela Flores Mora	\$15.200

Filtro por grado (ejemplo: Grado 9)

```
SELECT nombres || ' ' || apellidos AS nombre,
       grupo, deuda_actual
FROM Deudores
```

```
WHERE grado = 9
ORDER BY apellidos ASC;
```

Resultado — Solo grado 9		
Nombre	Grupo	Deuda
Camila Ortiz Herrera	02	\$6.100
Sebastián Vargas Díaz	01	\$50.000

Búsqueda por nombre (barra de búsqueda)

```
-- El usuario escribe 'var' en la barra de búsqueda
SELECT nombres || ' ' || apellidos AS nombre, deuda_actual
FROM Deudores
WHERE nombres LIKE '%var%'
      OR apellidos LIKE '%var%';
```

Resultado — Búsqueda 'var'	
Nombre encontrado	Deuda
Sebastián Vargas Díaz	\$50.000

Historial de movimientos de un estudiante

```
-- Ver todos los movimientos de Sebastián Vargas (id = 7)
SELECT tipo, monto, fecha
FROM Movimientos
WHERE id_deudor = 7
ORDER BY id_mov ASC;
```

Resultado — Historial de Sebastián Vargas Díaz		
Tipo	Monto	Fecha
 CREACION	\$47.800	13/03/2026 11:00
 CARGO	\$2.200	14/03/2026 08:00

5. Errores encontrados y cómo se resolvieron

Durante las pruebas aparecieron 6 errores. Todos se detectaron antes de integrar el código en la app, lo que evitó problemas mayores en producción.

Error 1

¿Qué pasó? Se podían insertar varios PINs.

Causa: La tabla App_Auth no impedía insertar más de un registro. Al reiniciar la app podían acumularse varios PINs y el sistema se confundía.

✓ Solución: Se controló desde Java: antes de insertar, el método existePin() verifica si ya hay un PIN guardado.

Error 2

¿Qué pasó? La deuda podía quedar en negativo.

Causa: Si el abono era mayor que la deuda, el resultado era un número negativo. Por ejemplo, deber \$3.000 y abonar \$5.000 daba -\$2.000.

✓ Solución: Se agregó la validación: if (nuevaDeuda < 0) nuevaDeuda = 0; en el método actualizarDeuda().

Error 3

¿Qué pasó? Se podía registrar el mismo estudiante dos veces.

Causa: Era posible ingresar 'carlos garcia' y 'Carlos García' como dos personas distintas por diferencia de mayúsculas.

✓ Solución: Se usó LOWER() en la consulta de validación: WHERE LOWER(nombres)=LOWER(?) AND LOWER(apellidos)=LOWER(?).

Error 4

¿Qué pasó? Al actualizar la app, el usuario perdía su PIN.

Causa: El método onUpgrade() borraba la tabla App_Auth completa al detectar una versión nueva del esquema.

✓ Solución: Se modificó onUpgrade() para nunca borrar App_Auth. Solo se borran Deudores y Movimientos, que se pueden volver a llenar.

Error 5

¿Qué pasó? La base de datos se quedaba 'abierta' en memoria.

Causa: Los métodos existePin() y validarPin() abrían la conexión a la base de datos pero nunca la cerraban, acumulando conexiones activas.

✓ Solución: Se agregó db.close() al final de cada método de lectura en DatabaseHelper.java.

Error 6

¿Qué pasó? Se registraba un movimiento de \$0 al liquidar una deuda ya pagada.

Causa: Si un estudiante ya tenía deuda \$0 y se ejecutaba liquidar, el sistema registraba un movimiento de \$0 en el historial.

✓ Solución: Se agregó la condición: if (deuda > 0) antes de registrar el movimiento de liquidación.

6. ¿Cuánto espacio ocupa la base de datos?

Una pregunta práctica: ¿cuántos estudiantes puede manejar la app antes de que el celular se ponga lento o se llene? Se hizo el cálculo para responderla.

Cada registro de deudor ocupa aproximadamente 200 bytes. Cada movimiento en el historial ocupa unos 110 bytes. Con eso:

Escenario real	Deudores	Movimientos aprox.	Tamaño total
Cafetería pequeña	50	250	~80 KB
Cafetería mediana	200	1.600	~350 KB
Cafetería grande	600	6.000	~1.2 MB
Límite recomendado	5.000	75.000	~12 MB

💡 Para el caso real de esta cafetería (entre 50 y 300 deudores activos), la base de datos pesará menos de 1 MB. Cualquier celular Android de los últimos 8 años maneja esto sin problema.

7. Resumen final

Se ejecutaron 15 pruebas en total. Todas terminaron en estado PASS tras las correcciones necesarias. El esquema pasó por 5 versiones antes de llegar al definitivo.

Prueba	¿Qué se verificó?	Resultado
Creación de tablas	Columnas, tipos de dato y restricciones correctas.	✓ PASS
Inserción de datos	INSERT con todos los campos, incluyendo campos opcionales.	✓ PASS
Duplicados bloqueados	Mismo nombre y apellido no se puede registrar dos veces.	✓ PASS
Deuda no negativa	Un abono mayor que la deuda deja el saldo en \$0, no negativo.	✓ PASS
Historial de movimientos	Cada CARGO, ABONO y LIQUIDACIÓN queda registrado con fecha.	✓ PASS
Búsqueda y filtros	LIKE, ORDER BY y filtro por grado devuelven datos correctos.	✓ PASS
Eliminar deudor	Al borrar un deudor, se eliminan también sus movimientos.	✓ PASS
Migración entre versiones	onUpgrade() conserva el PIN del usuario al actualizar.	✓ PASS
Rendimiento con 500 registros	Todas las consultas respondieron en menos de 2 segundos.	✓ PASS

Total pruebas	15
Errores encontrados	6
Versiones del esquema	5
Errores corregidos	6 de 6 (100%)
Pruebas aprobadas	15 de 15 (100%)